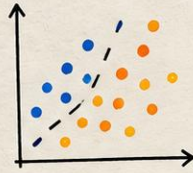


Data
to
Insights



- Classification
- Regression
- Clustering
- Dimensionality Reduction
- Model Selection

Build
Train
Evaluate
Improve
Repeat



Simple
Tools.
Powerful
Machine Learning.

scikit-learn

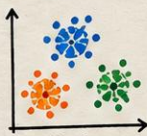
FULL NOTES

Same
Data.
More
Possibilities.

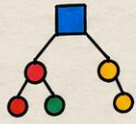


Learn
Apply
Grow

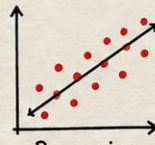
CLASSIC ALGORITHMS • PRACTICAL EXAMPLES • REAL-WORLD USE CASES



Cluster



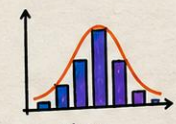
Decision Tree



Regression



Model Evaluation



More
Than Just Models!

Better
Models
Brighter
Decisions!



DataX Odyssey

ML Workflow & Scikit-learn Fundamentals

From data to decisions — made simple with Python!

"Same API.
Different problems.
Infinite possibilities." 😊

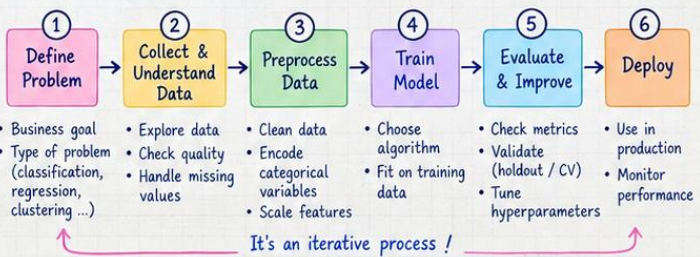
"Easy to learn
Powerful to build
Widely used
Open source" ❤️

1. What is Scikit-learn?

- Open-source Python library for machine learning.
- Simple and consistent API.
- Built on NumPy, SciPy, Matplotlib.
- Wide range of algorithms and tools for classification, regression, clustering, dimensionality reduction and more.



2. The ML Workflow



3. Core Modules

Module	Purpose
sklearn.datasets	Sample datasets, data loading utilities
sklearn.model_selection	Train-test split, cross-validation, hyperparameter tuning
sklearn.preprocessing	Scaling, encoding, imputation, etc.
sklearn.metrics	Model evaluation metrics
sklearn.pipeline	Pipelines for streamlined workflows
sklearn.compose	ColumnTransformer, FeatureUnion
sklearn.ensemble	Ensemble methods (bagging, boosting, etc.)
sklearn.decomposition	PCA, TruncatedSVD, etc.
sklearn.cluster	Clustering algorithms
sklearn.neighbors	KNN and nearest neighbors
sklearn.linear_model	Linear and logistic regression
sklearn.tree	Decision trees
sklearn.svm	Support vector machines

4. The Estimator API

- All models in scikit-learn follow a consistent and simple API.
- Makes it easy to learn, switch between models and build pipelines.

Common methods:

- `fit()` → train the model
- `predict()` → make predictions
- `transform()` → transform data (e.g., scaling, PCA)
- `fit_predict()` → fit and predict together (e.g., clustering)
- `score()` → return default performance metric

✓ Same interface works across different algorithms!

5. A Simple Example

```
# Import libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Preprocess (scaling)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
model = LogisticRegression()
model.fit(X_train, y_train)

# Predict & evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Just a few lines to build a working ML model! 😊

6. Key Features

- ✓ Clean and consistent API
- ✓ Excellent documentation
- ✓ Wide range of algorithms
- ✓ Seamless integration with other Python libraries
- ✓ Active community support
- ✓ Suitable for both beginners and professionals
- ✓ Used in research and industry

Focus on solving problems, not complex infrastructure.

7. Why Use It?

- ⚙️ Easy to learn and use
- 🕒 Saves development time
- 📊 Reliable and well-tested
- 👥 Large community and resources
- 💡 Great for prototyping and production

8. Popular Datasets (for practice)

Dataset	Description
<code>load_iris()</code>	Iris flower classification (small)
<code>load_boston()</code>	Housing price regression (deprecated, use alternatives)
<code>load_digits()</code>	Handwritten digit classification
<code>load_wine()</code>	Wine classification
<code>load_breast_cancer()</code>	Cancer classification

Explore. Experiment. Build. Learn. 😊

Data Preprocessing

Clean data → Better models → Real insights

"Good data.
preprocessing
turns data into
opportunities." 😊

Same data.
Different
preprocessing.
Different results!

1. Handling Missing Values

- Missing values can bias the model and reduce performance.
- Use SimpleImputer to fill missing values.
- Common strategies: mean, median, most_frequent, constant.

```
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)
```

Before		
Age	Income	City
25	50000	Delhi
NaN	60000	Mumbai
35	NaN	Delhi

After (median)		
Age	Income	City
25	50000	Delhi
30	60000	Mumbai
35	60000	Delhi

No missing values,
no missed opportunities!

2. Encoding Categorical Variables

- Convert categorical data into numerical format.
- Use OneHotEncoder for nominal variables.
- Use OrdinalEncoder for ordinal variables.

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(drop='first',
                        sparse_output=False)
X_encoded = encoder.fit_transform(X_cat)
```

Original	
City	Gender
Delhi	Male
Mumbai	Female
Bangalore	Male

One-Hot Encoded		
City_Delhi	City_Mumbai	Gender_Male
1	0	1
0	1	0
0	0	1

Categories → Numbers → Models understand!

3. Feature Scaling

- Brings all features to a similar scale.
- Helps many algorithms (KNN, SVM, Logistic Regression, PCA, etc.).
- Common methods: StandardScaler, MinMaxScaler, RobustScaler.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Before Scaling	
Age	Income
25	50000
35	60000
45	80000

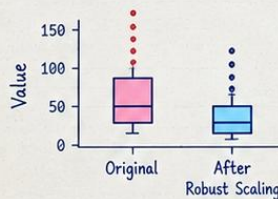
After Scaling	
Age	Income
-1.22	-1.07
0.00	0.00
1.22	1.07

Different scales, same importance!

4. Handling Outliers

- Outliers can affect model performance.
- Use RobustScaler (less sensitive to outliers).
- Or cap values using domain knowledge (e.g., IQR method).

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)
```



Outliers happen.
Be robust!

5. Train/Test Split (Quick Reminder)

- Split data into train and test sets before preprocessing (or use pipelines).
- Helps evaluate generalization.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Split early.
Avoid data leakage! 😊

6. Useful Preprocessing Classes

Class	Purpose
SimpleImputer	Fill missing values
StandardScaler	Standardize features (mean=0, std=1)
MinMaxScaler	Scale features to [0, 1]
RobustScaler	Scale features (robust to outliers)
OneHotEncoder	Encode nominal categorical variables
OrdinalEncoder	Encode ordinal categorical variables
PolynomialFeatures	Generate polynomial and interaction features
Binarizer	Convert features to binary (0/1)
FunctionTransformer	Apply custom transformations

"Preprocessing is not just a step,
it's the foundation." 💡

7. Putting It Together (Pipeline Preview)

- Use a pipeline to apply preprocessing steps in the right order.
- Prevents data leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
pipeline.fit(X_train, y_train)
```

Clean → Transform → Model → Predict! ⚙️

Key Takeaways

- ✓ Handle missing values appropriately.
- ✓ Encode categorical variables.
- ✓ Scale features when needed.
- ✓ Treat outliers with care.
- ✓ Use pipelines for consistent preprocessing.
- ✓ Preprocessing improves model performance and stability.
- ✓ Choose the right technique based on data and algorithm.

Clean Data.
Better Models.
Brighter Insights. 😊

Same data.
More possibilities! 📊

DataXOdyssey

Train / Test Split & Cross-Validation

Evaluate better. Generalize to new data.

Good validation today, better predictions tomorrow! 😊

Same data. Different split. Different insights.



1. Why Split the Data ?

- To evaluate how well the model generalizes to unseen data.
- Helps prevent overfitting.
- Gives an unbiased estimate of model performance.
- Standard practice: 70/30 or 80/20.

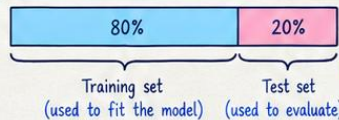
“Train to learn, Test to trust.”



2. Train / Test Split

- Splits the data into training and test sets (randomly by default).
- Use `train_test_split()` from `model_selection`.
- Common parameters: `test_size`, `random_state`, `stratify` (for classification).

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```



Use `stratify=y` for classification to maintain class distribution!

3. Important Parameters

Parameter	Description
<code>test_size</code>	Proportion of test set (float) or number of samples (int)
<code>train_size</code>	Proportion of train set (float) or number of samples (int)
<code>random_state</code>	Ensures reproducible results
<code>shuffle</code>	Whether to shuffle before splitting (default=True)
<code>stratify</code>	Preserves class distribution (important for classification)

4. Cross-Validation (CV)

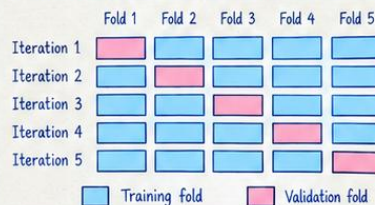
- A more robust way to evaluate performance.
- Splits the training data into K folds.
- Trains on K-1 folds and validates on the remaining fold.
- Reduces variance from a single train/test split.
- Commonly used: K-Fold or Stratified K-Fold.

“More than one split, a clearer picture!”

5. K-Fold Cross-Validation

- Splits data into K equal parts (folds).
- Each fold gets a chance to be the validation set.
- Returns the average performance.

```
from sklearn.model_selection import KFold, cross_val_score
kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=kf, scoring='accuracy')
print(f"CV Accuracy: {scores.mean():.4f} (+/- {scores.std():.4f})")
```

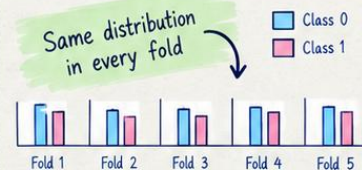


6. Stratified K-Fold

- Maintains the same class distribution in each fold.
- Recommended for classification problems.

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(model, X, y, cv=skf, scoring='f1')
print(f"CV F1-score: {scores.mean():.4f}")
```

Same distribution in every fold



7. When to Use What ?

Scenario	Recommended
Small dataset	Cross-validation (K-Fold)
Classification problem	Stratified K-Fold
Imbalanced classes	Stratified K-Fold + proper metrics (e.g., F1, ROC-AUC)
Large dataset	Train/Test split (faster)
Hyperparameter tuning	Cross-validation (used in GridSearchCV)

No data leakage!

- ✓ Split before preprocessing.
- ✓ Fit preprocessors on training data only.
- ✓ Use pipelines (more on this later!).



“Validate well. Build with confidence.”

Pipelines & ColumnTransformer

Cleaner code. Consistent preprocessing. No data leakage.

"Preprocessing + Model in one workflow = Better & Safer ML." 😊

1. Why Use a Pipeline ?

- Chains multiple steps (preprocessing + model) into a single workflow.
- Ensures the same transformations are applied during training and testing.
- Reduces the risk of data leakage.
- Cleaner and more maintainable code.
- Works with cross-validation and hyperparameter tuning.

Same steps.
Every time.
No leaks!



2. Basic Pipeline

Pipeline takes a list of steps (name, transformer) and runs them in sequence.

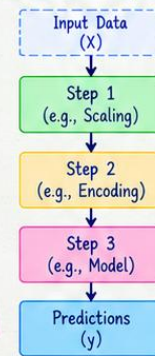
```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression())
])
```

```
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

Define steps
→ Fit
→ Predict
Simple! 😊

3. How Pipeline Works ?



During predict(), the same transformations learned on training data are applied to new data automatically! 😊

4. Introducing ColumnTransformer

- Apply different preprocessing steps to different columns.
- Very useful for datasets with mixed data types (numerical + categorical).
- Combines multiple transformers in parallel.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])
```

Different columns.
Different steps.
One transformer! 😊

5. Example: Pipeline + ColumnTransformer

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import RandomForestClassifier
```

```
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_features)
])
```

```
model = Pipeline([
    ('preprocess', preprocessor),
    ('model', RandomForestClassifier())
])
```

```
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

Preprocess + Model
in one clean pipeline! 💡

6. Example Dataset Structure

Age	Income	City	Gender	Target
25	50000	Delhi	Male	1
32	62000	Mumbai	Female	0
28	58000	Bangalore	Male	1
40	72000	Delhi	Female	0

Numerical
(Scale)

Categorical
(One-Hot Encode)

7. Useful Functions & Tips

- ✓ Use Pipeline for consistent preprocessing.
- ✓ Use ColumnTransformer for mixed data types.
- ✓ Set `handle_unknown='ignore'` for new categories.
- ✓ Works seamlessly with GridSearchCV.
- ✓ Use meaningful step names.
- ✓ Check feature names with `get_feature_names_out()`.

8. Key Takeaways

- ✓ Pipeline helps chain preprocessing and modeling steps.
- ✓ ColumnTransformer applies different transformations to different columns.
- ✓ Prevents data leakage and ensures reproducibility.
- ✓ Makes code cleaner and easier to maintain.
- ✓ Essential for real-world datasets.

Clean data.
Clean pipeline.
Better models
🚀 😊

"Good ML isn't just about the model, but the pipeline that gets you there."

Model Evaluation

Measure → Compare → Choose better models

"A good metric turns model output into meaningful decisions." 😊

Same data.
Different metrics.
Different story!



1. Classification Metrics

- Used when target is categorical (e.g., 0/1 or multi-class).
- Common metrics: Accuracy, Precision, Recall, F1 Score, ROC-AUC.

```
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix,
    roc_auc_score
)
```

2. Confusion Matrix

- Shows actual vs predicted values.
- Helps understand types of errors (TP, FP, TN, FN).

		Predicted	
		Positive (1)	Negative (0)
Actual	Positive (1)	TP (True Positive)	FN (False Negative)
	Negative (0)	FP (False Positive)	TN (True Negative)

3. Key Classification Metrics

Metric	Formula (Binary)	Meaning
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	Overall correct predictions
Precision	$TP / (TP + FP)$	Of predicted positive, how many are correct?
Recall (Sensitivity)	$TP / (TP + FN)$	Of actual positive, how many are detected?
F1 Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	Balance between precision and recall
ROC-AUC	Area under ROC curve	Measures separability (higher is better)

4. Classification Report

- Gives precision, recall, f1-score and support for each class.
- Useful for multi-class problems too.

```
from sklearn.metrics import classification_report
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

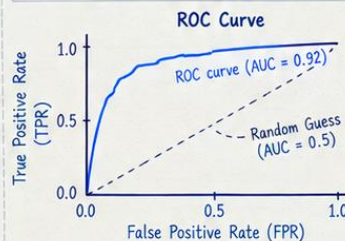
	precision	recall	f1-score	support
0	0.92	0.88	0.90	120
1	0.85	0.90	0.87	100
accuracy			0.89	220
macro avg	0.89	0.89	0.89	220
weighted avg	0.89	0.89	0.89	220

Look beyond accuracy
— check per-class performance! 😊

5. ROC Curve & AUC

- Plots True Positive Rate (TPR) vs False Positive Rate (FPR) at different thresholds.
- AUC closer to 1 is better.

```
from sklearn.metrics import roc_curve, roc_auc_score
probs = model.predict_proba(X_test)[:, 1]
fpr, tpr, _ = roc_curve(y_test, probs)
auc = roc_auc_score(y_test, probs)
```



6. Regression Metrics

- Used when target is continuous (e.g., house price, sales).
- Common metrics: MAE, MSE, RMSE, R².

```
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error,
    r2_score
)
y_pred = model.predict(X_test)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", mean_squared_error(y_test, y_pred,
    squared=False))
print("R2 Score:", r2_score(y_test, y_pred))
```

Metric	Formula	Meaning
MAE	$\frac{1}{n} \sum y - \hat{y} $	Average absolute error
MSE	$\frac{1}{n} \sum (y - \hat{y})^2$	Penalizes larger errors
RMSE	$\sqrt{\text{MSE}}$	Same unit as target
R ²	$1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$	Proportion of variance explained (higher is better)

7. Cross-Validation Score

- Gives a more reliable estimate of performance.
- Reduces dependence on a single train/test split.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print("CV Accuracy:", scores.mean(), "+/-", scores.std())
```

More robust estimates.
Less variance. Better trust! 📊

8. Choosing the Right Metric

- Classification → Accuracy, F1, ROC-AUC (depending on problem).
- Regression → MAE, RMSE, R².
- Imbalanced data → Prefer Precision, Recall, F1, ROC-AUC.
- Business context → Choose metric that aligns with the goal.

There is no single "best" metric.
Choose what solves your problem! 😊

9. Key Takeaways

- ✓ Use the right metric for the problem type.
- ✓ Look at multiple metrics, not just one.
- ✓ Confusion matrix helps understand errors.
- ✓ Cross-validation gives reliable estimates.
- ✓ For imbalanced data, prefer Precision, Recall, F1 or ROC-AUC.
- ✓ Align the metric with business goals.

Evaluate better.
Build smarter. 💡

Hyperparameter Tuning & Model Selection

Find the best model. Get the best performance.

"Same model.
Different
hyperparameters.
Big difference
in performance!" 😊

Tune.
Compare.
Choose.
Do Better.



1. What are Hyperparameters?

- Settings you specify before training the model.
- Control how the model learns.
- Different values can significantly affect performance.
- Examples: learning_rate, n_estimators, max_depth, C, alpha, etc.

Not learned from data,
but chosen by you! 💡

2. Manual Tuning (Basic Idea)

- Try different values and check performance.
- Time-consuming and not efficient.
- Useful for quick experimentation.

```
model = RandomForestClassifier(n_estimators=100,  
                             max_depth=10)  
model.fit(X_train, y_train)  
preds = model.predict(X_test)
```

Works, but not scalable
for many parameters! 😊

3. GridSearchCV

- Tries all possible combinations of parameter values.
- Uses cross-validation to find the best model.
- Good when the parameter space is small.

```
from sklearn.model_selection import GridSearchCV  
param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [None, 5, 10],  
    'min_samples_split': [2, 5]  
}  
clf = GridSearchCV(RandomForestClassifier(),  
                  param_grid, cv=5, scoring='accuracy')  
clf.fit(X_train, y_train)
```

4. RandomizedSearchCV

- Tries a random combination of parameter values (not all).
- Faster than GridSearchCV.
- Good when the parameter space is large.

```
from sklearn.model_selection import RandomizedSearchCV  
from scipy.stats import randint  
param_dist = {  
    'n_estimators': randint(50, 300),  
    'max_depth': [None, 5, 10, 15],  
    'min_samples_split': randint(2, 10)  
}  
clf = RandomizedSearchCV(RandomForestClassifier(),  
                        param_dist, n_iter=20, cv=5,  
                        scoring='accuracy', random_state=42)  
clf.fit(X_train, y_train)
```

5. Comparing Models

- Use cross-validation to compare different models.
- Helps choose the best algorithm, not just best parameters.

```
from sklearn.model_selection import cross_val_score  
from sklearn.linear_model import LogisticRegression  
from sklearn.ensemble import RandomForestClassifier  
  
models = {  
    'Logistic Regression': LogisticRegression(),  
    'Random Forest': RandomForestClassifier(),  
}  
  
for name, model in models.items():  
    scores = cross_val_score(model, X, y, cv=5,  
                            scoring='accuracy')  
    print(f"{name}: {scores.mean():.3f} (+/- {scores.std():.3f})")
```

6. Results & Best Model

- After tuning, check the best parameters and best estimator.
- Use the best model for final predictions.

```
print("Best parameters:", clf.best_params_)  
print("Best cross-validation score:",  
      clf.best_score_)  
  
best_model = clf.best_estimator_  
preds = best_model.predict(X_test)
```

Best parameters =
better performance! 😊



7. Common Hyperparameters (Examples)

Algorithm	Common Hyperparameters
Linear Regression	fit_intercept, positive
Logistic Regression	C, penalty, solver, max_iter
Decision Tree	max_depth, min_samples_split, min_samples_leaf
Random Forest	n_estimators, max_depth, min_samples_split
SVM	C, kernel, gamma
KNN	n_neighbors, weights, metric
K-Means	n_clusters, init, max_iter
Gradient Boosting	n_estimators, learning_rate, max_depth, subsample

8. Tips for Effective Tuning

- ✓ Start with a small parameter grid.
- ✓ Use RandomizedSearchCV for large spaces.
- ✓ Always use cross-validation.
- ✓ Choose the right evaluation metric (e.g., accuracy, F1, ROC-AUC, RMSE).
- ✓ Consider computational cost.
- ✓ Analyze results (e.g., cv_results_).
- ✓ Finally, evaluate on a separate test set.

Tune smart.
Not just harder. 😊

9. Key Takeaways

- ✓ Hyperparameters control model behavior.
- ✓ GridSearchCV exhaustively searches.
- ✓ RandomizedSearchCV is faster and efficient.
- ✓ Cross-validation helps in reliable model selection.
- ✓ Compare multiple models, not just parameters.
- ✓ The best model balances performance, complexity and business needs.

Better tuning.
Better models.
Real impact.



Same data.
Better decisions. 😊

DataXOdyssey

Feature Engineering & Selection

Better features → Better models → Better insights

"Good features make simple models powerful." 😊

Same data. Better features. Better results.



1. What is Feature Engineering?

- Create new features or transform existing ones to help the model learn better.
- Can improve model performance significantly.
- Includes domain knowledge, transformations and feature extraction.

Good features = simpler models + better performance!



2. Common Transformations

- Create new features from existing ones.
- Handle non-linear relationships.
- Examples: polynomial features, log transformation, binning, interaction terms.

```
from sklearn.preprocessing import PolynomialFeatures
import numpy as np

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

Creates original features + polynomial terms (e.g., x , x^2 , $x_1 \times x_2$, etc.)

3. Encoding Categorical Features

- Convert categorical variables into numbers so that ML models can use them.
- Common methods: One-Hot Encoding, Ordinal Encoding, Target Encoding (advanced).

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
X_cat = encoder.fit_transform(X[['city']])
```

Original	One-Hot Encoded		
City	Delhi	Mumbai	Bangalore
Delhi	1	0	0
Mumbai	0	1	0
Bangalore	0	0	1

4. Feature Scaling

- Brings features to a similar scale.
- Important for distance-based algorithms (e.g., KNN, SVM) and regularized models.
- Common methods: StandardScaler, MinMaxScaler, RobustScaler.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Feature	Before	After (StandardScaler)
Age	25	-1.21
Income	50000	0.34
Balance	120000	1.18

Same feature. Different scale. Different impact!



5. Feature Selection Methods

- Select the most relevant features.
- Helps reduce overfitting, training time and improves model interpretability.
- Common methods: Filter, Wrapper, Embedded.

5.1 Filter Methods (e.g., SelectKBest)

```
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif

selector = SelectKBest(score_func=f_classif, k=5)
X_new = selector.fit_transform(X, y)
```

Selects top K features based on statistical test (e.g., ANOVA F-value).

5.2 Wrapper Methods (e.g., RFE)

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

rfe = RFE(estimator=LogisticRegression(), n_features_to_select=5)
X_rfe = rfe.fit_transform(X, y)
```

Recursively selects features by training the model.

5.3 Embedded Methods (e.g., L1 Regularization)

```
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X, y)
selected_features = X.columns[model.coef_ != 0]
```

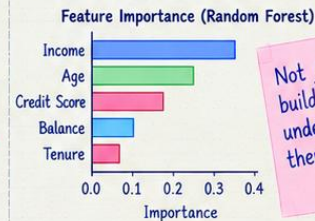
Feature selection happens during model training.

6. Feature Importance

- Many models provide feature importance scores (e.g., tree-based models).
- Helps understand which features contribute most to predictions.

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X, y)
importances = model.feature_importances_
```



Not just build models, understand them! 😊

7. Feature Engineering Ideas

- ✓ Create interaction features (e.g., age x income).
- ✓ Extract date/time features (year, month, day, etc.).
- ✓ Bin continuous variables (e.g., age groups).
- ✓ Use domain knowledge (e.g., debt-to-income ratio).
- ✓ Handle missingness as a feature.
- ✓ Text features (e.g., TF-IDF, length, word count).

Be creative. Domain knowledge makes the difference! 😊

8. Key Takeaways

- ✓ Good features can significantly improve model performance.
- ✓ Use the right transformation and selection technique for your problem.
- ✓ Combine domain knowledge with data-driven methods.
- ✓ Remove irrelevant or redundant features.
- ✓ Always validate the impact of feature engineering using cross-validation.

9. Remember

Better features → Simpler models → Stronger results → Real business impact. 😊

"Features are the foundation. Models just build on top."



Ensemble Learning

Scikit-learn in Practice

Combine multiple models → Better predictions → More robust results

Different models.
Same goal.
Stronger predictions! 😊

Ensemble
> Individual
(most of the time)



1. Why Ensemble Learning?

- Combines multiple models to make better predictions.
- Reduces variance and/or bias.
- More robust to overfitting.
- Often gives higher accuracy than a single model.

"Many learners,
one stronger model!"

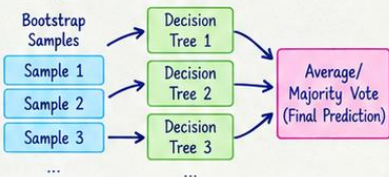


2. Types of Ensemble Methods

Type	Idea	Examples (in sklearn)
Bagging	Train multiple models in parallel on different samples and average.	RandomForest, BaggingClassifier, BaggingRegressor
Boosting	Train models sequentially, each corrects previous errors.	AdaBoost, GradientBoosting
Stacking	Combine predictions of multiple models using a meta-model.	StackingClassifier, StackingRegressor
Voting	Combine predictions from different models (e.g., majority vote).	VotingClassifier, VotingRegressor

3. Bagging (e.g., Random Forest)

- Train multiple models on different bootstrap samples (with replacement).
- Final prediction = average (regression) or majority vote (classification).
- Reduces variance, good for complex models (e.g., decision trees).



4. Random Forest (Example)

- Ensemble of decision trees using bagging.
- Works well out of the box.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    random_state=42
)
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

Simple to use.
Powerful results! 😊



5. Boosting (e.g., AdaBoost)

- Train models sequentially, each model focuses on the errors of the previous one.
- Reduces bias.
- Works well with weak learners (e.g., decision trees).

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
base = DecisionTreeClassifier(max_depth=1)
model = AdaBoostClassifier(
    estimator=base,
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
model.fit(X_train, y_train)
```

6. Gradient Boosting (Example)

- Optimizes a loss function using gradient descent.
- Often gives high performance.
- Variants: GradientBoosting, HistGradientBoosting.

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

Small steps.
Big improvement! 😊



7. Voting (Combine Different Models)

- Combine predictions from different models (e.g., logistic regression + random forest).
- Supports soft voting (probabilities) and hard voting (class labels).

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
lr = LogisticRegression()
rf = RandomForestClassifier()
model = VotingClassifier(
    estimators=[('lr', lr), ('rf', rf)],
    voting='soft' # or 'hard'
)
model.fit(X_train, y_train)
```

Different models.
Better decisions! 😊

8. Stacking (Use a Meta-Model)

- Train multiple base models and combine their predictions using a meta-model.
- Can capture complex patterns.

```
from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
estimators = [
    ('rf', RandomForestClassifier()),
    ('svm', SVC(probability=True))
]
model = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
model.fit(X_train, y_train)
```

9. Key Takeaways

- ✓ Ensembles are usually more robust than single models.
- ✓ Bagging → reduces variance (e.g., Random Forest).
- ✓ Boosting → reduces bias (e.g., AdaBoost, Gradient Boosting).
- ✓ Voting → simple way to combine different models.
- ✓ Stacking → uses a meta-model for better predictions.
- ✓ Tune key parameters: n_estimators, learning_rate, max_depth, etc.
- ✓ Try multiple methods and compare using cross-validation.

Same data.
Stronger models.
Better outcomes! 😊

Unsupervised Learning with Scikit-learn

Find patterns. Explore structure. Gain insights.

"No labels.
Still powerful.
Discover what
your data is telling
you!"

Same data.
Different view.
Bigger insights.

1. What is Unsupervised Learning?

- Learns from unlabeled data.
- Finds hidden patterns and structure.
- Common tasks: clustering, dimensionality reduction, anomaly detection, association rules.
- Useful for exploration, segmentation and feature engineering.

"No target variable,
but plenty to learn!"



2. K-Means Clustering

- Partitions data into K clusters.
- Simple, fast and widely used.
- Works well for spherical clusters.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

kmeans = KMeans(n_clusters=3,
                 random_state=42)
labels = kmeans.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels,
            cmap='Set2', s=50)

plt.scatter(kmeans.cluster_centers_[0],
            kmeans.cluster_centers_[1],
            c='black', marker='X', s=200)

plt.title('K-Means Clustering')
plt.show()
```



Simple algorithm.
Powerful insights!

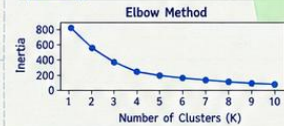
3. Choosing the Right K

- Use the Elbow Method to find optimal number of clusters.
- Look for the "elbow" point where inertia stops decreasing significantly.

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
K = range(1, 11)
for k in K:
    kmeans = KMeans(n_clusters=k,
                    random_state=42)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)

plt.plot(K, inertia, marker='o')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```



Look for
the elbow!

4. DBSCAN Clustering

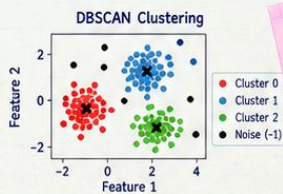
- Finds clusters of arbitrary shape.
- Identifies noise (outliers).
- Works well when clusters have different densities.

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.5, min_samples=5)
labels = dbscan.fit_predict(X)

plt.scatter(X[:, 0], X[:, 1], c=labels,
            cmap='Set1', s=50)

plt.title('DBSCAN Clustering')
plt.show()
```



Finds
clusters +
outliers!

5. Clustering Evaluation

- Use metrics to evaluate cluster quality (when ground truth is not available).
- Common metrics: Silhouette Score, Davies-Bouldin Index, Calinski-Harabasz Index.

```
from sklearn.metrics import silhouette_score,
daviess_bouldin_score, calinski_harabasz_score

sil_score = silhouette_score(X, labels)
db_score = daviess_bouldin_score(X, labels)
ch_score = calinski_harabasz_score(X, labels)

print('Silhouette Score: ', sil_score)
print('Davies-Bouldin Index: ', db_score)
print('Calinski-Harabasz Index: ', ch_score)
```

Metric	Range	Interpretation
Silhouette Score	$[-1, 1]$	Higher is better
Davies-Bouldin	$[0, \infty)$	Lower is better
Calinski-Harabasz	$[0, \infty)$	Higher is better

Use multiple metrics.
There's no single best measure!



6. PCA (Dimensionality Reduction)

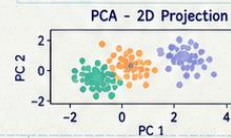
- Reduces the number of features.
- Helps in visualization and removes noise.
- Transforms data into uncorrelated components (principal components).

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:, 0], X_pca[:, 1],
            c=labels, cmap='Set2', s=50)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA - 2D Projection')
plt.show()
```



Less
dimensions.
More clarity!

7. Common Unsupervised Models in Scikit-learn

Task	Model(s)	Key Parameters
Clustering	KMeans	n_clusters, init, random_state
	DBSCAN	eps, min_samples
Hierarchical Clustering	AgglomerativeClustering	n_clusters, linkage, affinity
Dimensionality Reduction	PCA	n_components, svd_solver
Anomaly Detection	IsolationForest	contamination, n_estimators
Association Rules	(via mlxtend)	apriori, association_rules

8. Key Takeaways

- ✓ Unsupervised learning helps explore and understand data.
- ✓ K-Means is simple and effective for well-separated clusters.
- ✓ DBSCAN handles complex shapes and outliers.
- ✓ Use evaluation metrics to assess cluster quality.
- ✓ PCA helps reduce dimensions and visualize data.
- ✓ Choose the right algorithm based on data characteristics.

Explore. Visualize. Understand.
That's the power of unsupervised learning!

9. Real-World Applications

- Customer segmentation (e.g., retail, banking)
- Image compression (PCA)
- Anomaly detection (e.g., fraud, network security)
- Document clustering (e.g., topic modeling)
- Recommendation systems (preprocessing step)
- Exploratory data analysis

Same data.
New Perspectives.
Real Impact!



DataXOdyssey

From data to decisions! 😊

Scikit-learn

End-to-End Cheat Sheet

"Same workflow. Different problems. Real results!" 😊

Build. Evaluate. Improve. Repeat.



1. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split,
    cross_val_score, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import *
from sklearn.ensemble import RandomForestClassifier
# (import other models as needed)
```

Start here!

2. Load & Explore Data

```
df = pd.read_csv('data.csv')

print(df.shape)
df.head()
df.info()
df.describe()
df.isnull().sum()
```

Understand your data!



3. Preprocess Data

3.1 Train/Test Split

```
X = df.drop('target', axis=1)
y = df['target']

X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.2,
                    random_state=42,
                    stratify=y)
```

3.2 Preprocessing (Numeric + Categorical)

```
numeric_features = ['age', 'income']
categorical_features = ['city', 'gender']

numeric_transformer = Pipeline([
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline([
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer, numeric_features),
    ('cat', categorical_transformer, categorical_features)
])
```

4. Build a Model

4.1 Using a Pipeline

```
from sklearn.linear_model import
    LogisticRegression

model = Pipeline([
    ('preprocess', preprocessor),
    ('model', LogisticRegression(
        max_iter=1000, random_state=42))
])
```

4.2 Train the Model

```
model.fit(X_train, y_train)
```

Clean pipeline. Less errors. Better results! 😊

5. Make Predictions

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

From data to predictions!



6. Evaluate the Model

```
print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
from sklearn.metrics import roc_auc_score
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

Choose the right metrics for your problem! 😊

Common Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC
- MSE / RMSE (Regression)
- Silhouette Score (Clustering)

7. Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'model_C': [0.01, 0.1, 1, 10],
    'model_penalty': ['l2'],
    'model_solver': ['lbfgs']
}

grid = GridSearchCV(model, param_grid,
                    cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)

print("Best Params:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

Find the best model! 🔍

8. Cross-Validation

```
scores = cross_val_score(model, X, y,
                        cv=5, scoring='accuracy')

print("CV Accuracy:", scores.mean())
print("Std Dev:", scores.std())
```

More reliable performance estimate! 😊

9. Common Models (import & use)

Task	Model(s)	Import
Classification	Logistic Regression	from sklearn.linear_model import LogisticRegression
	Random Forest	from sklearn.ensemble import RandomForestClassifier
	SVM, KNN, Naive Bayes	from sklearn.svm import SVC from sklearn.neighbors import KNeighborsClassifier from sklearn.naive_bayes import GaussianNB
Regression	Linear Regression	from sklearn.linear_model import LinearRegression, Ridge, Lasso
	Ridge, Lasso	
	Random Forest	from sklearn.ensemble import RandomForestRegressor
Clustering	K-Means, DBSCAN, Hierarchical	from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
Dimensionality Reduction	PCA	from sklearn.decomposition import PCA

10. Save & Deploy

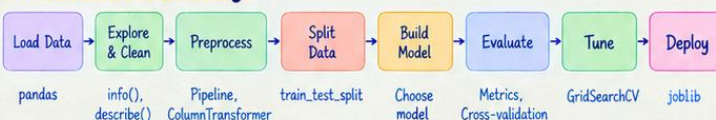
```
import joblib

joblib.dump(model, 'model.pkl')

# Load later
model = joblib.load('model.pkl')
```

Train once. Use anywhere! 😊

11. Workflow Summary



Learn. Apply. Grow. 😊

DataXOdyssey

12. Key Takeaways

- ✓ Follow a consistent workflow.
- ✓ Use Pipelines to keep preprocessing + modeling together.
- ✓ Choose the right evaluation metric for your problem.
- ✓ Validate with cross-validation.
- ✓ Tune hyperparameters for better performance.
- ✓ Scikit-learn is simple, consistent and powerful.
- ✓ Same workflow works for classification, regression and clustering.
- ✓ Practice on real datasets!

Machine Learning made simpler! 😊